

Rapport de Projet – Soutenance Finale « SPECTRALIS »

Participants :

Florent Delbey
Oren Guetta
Tao Brouta-Besset
Julien Dos Santos

EPITA Promo 2029

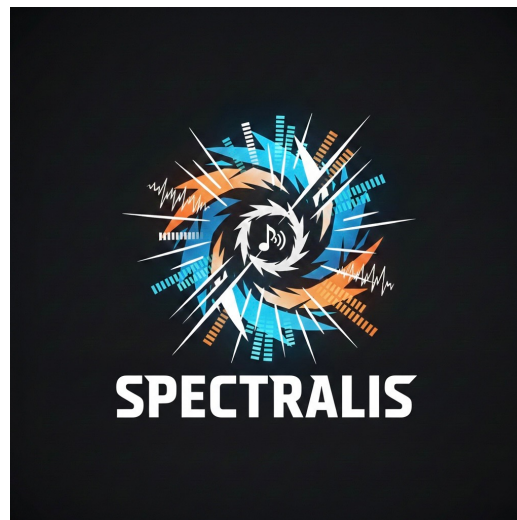


Table des matières

1	Rôle et mission de chaque membre	2
1.1	Florent Delbey (Chef de Groupe & Analyse Spectrale)	2
1.2	Oren Guetta	2
1.3	Tao Brouta-Besset	2
1.4	Julien Dos Santos	3
2	Objectifs finaux, Bilan des tâches et État d'avancement	4
3	Difficultés & Choix technologiques et Algorithmiques	7
3.1	Analyse Spectrale - Florent	7
3.2	Synthèse Granulaire - Oren	8
3.3	Entrées/Sorties Audio & Pipeline - Tao	9
3.4	Système & Rendu - Julien	11
4	Livrables Finaux et Historique de Développement	12
5	Bibliographie Détaillée du Projet	12

1 Rôle et mission de chaque membre

1.1 Florent Delbey (Chef de Groupe & Analyse Spectrale)

En tant que chef de projet, j'ai assuré la coordination de l'équipe, la planification des tâches et l'intégration continue de nos modules tout au long du semestre. Sur le plan technique, j'ai été le développeur principal du moteur d'analyse spectrale. Ma mission a consisté à concevoir et implémenter l'algorithme de Transformée de Fourier Rapide (FFT) en Rust permettant de passer du signal temporel au domaine fréquentiel en temps réel, afin d'alimenter le rendu visuel et de guider les modulations du moteur.

1.2 Oren Guetta

Au sein du projet SPECTRALIS, j'interviens en tant qu'Expert Traitement de Signal & Synthèse Granulaire, ainsi que responsable de l'intégration continue. Ma mission principale est de concevoir le moteur de synthèse, qui constitue le cœur créatif du logiciel. Je suis chargé de transformer un flux audio continu en une texture granulaire malléable, en découpant le signal en micro-fragments (grains) pour permettre des manipulations temporelles et fréquentielles. Au-delà de l'aspect algorithmique, mon rôle a consisté à concevoir l'architecture de données centrale du moteur : un tampon circulaire (Circular Buffer) "lock-free" basé sur des variables atomiques. Cette structure garantit le transfert sécurisé des données audio entre le thread du microphone et celui de l'interface graphique, sans jamais bloquer l'exécution temps réel. Enfin, j'ai pris en charge la centralisation du dépôt Git et veillé à la stricte conformité du code avec les exigences du projet (zéro warning, typage strict, portabilité de la compilation sous Linux).

1.3 Tao Brouta-Besset

Au sein du projet SPECTRALIS, j'interviens en tant que responsable de la gestion des entrées/sorties audio et de l'architecture du pipeline temps réel. Ma mission principale est de concevoir l'infrastructure reliant la carte son, les sources audio et les différents modules de traitement du logiciel, afin d'assurer une circulation fluide, stable et exploitable du signal dans l'ensemble de l'application. Au-delà de l'aspect purement technique, mon

rôle a consisté à structurer le moteur audio autour du modèle en callback imposé par la bibliothèque `cpal`. J'ai pris en charge la configuration des périphériques d'entrée et de sortie, la gestion des flux provenant du microphone et des fichiers audio, ainsi que leur intégration dans le moteur granulaire. J'ai également mis en place la logique de resampling, l'enregistrement audio, la lecture des buffers enregistrés et l'alimentation continue du buffer de traitement, de manière à maintenir un comportement cohérent quelles que soient les sources utilisées. Enfin, j'ai assuré la communication entre le thread principal et le thread audio grâce à une architecture fondée sur des variables atomiques et des structures partagées limitées aux cas strictement nécessaires. Cette organisation permet de piloter les paramètres sans bloquer l'exécution temps réel. Elle a aussi permis d'intégrer le calcul de la FFT dans le pipeline de sortie afin d'alimenter l'affichage spectral en temps réel, tout en garantissant une exécution stable, thread-safe et compatible avec les contraintes de latence du projet.

1.4 Julien Dos Santos

Au sein du projet *Spectralis*, mon rôle principal a été le développement de l'interface graphique native en Rust, la conception et la mise à jour du site web de présentation, ainsi que la coordination entre les différents pôles techniques de l'équipe.

Sur le plan de l'interface graphique, j'ai utilisé la bibliothèque `egui` avec le backend `eframe` et le renderer `Glow` (OpenGL), ce qui m'a permis de construire une interface entièrement intégrée au sein de l'application Rust, sans dépendance externe à un framework UI séparé. L'interface expose en temps réel tous les paramètres du moteur granulaire : taille de grain, densité, pitch, mode source (micro ou fichier), transport audio, enregistrement et lecture. Elle intègre également un analyseur de spectre FFT temps réel, alimenté par les données produites par le module FFT développé par un autre membre de l'équipe.

Sur le plan du site web, j'ai conçu et développé l'intégralité de la vitrine du projet, de la première soutenance jusqu'à la version finale présentée aujourd'hui, en veillant à ce qu'elle reflète fidèlement l'état d'avancement réel du projet à chaque étape.

Enfin, sur le plan de la coordination, j'ai assuré le lien technique entre les modules développés indépendamment, notamment l'intégration du pipeline FFT dans le callback audio et sa connexion à l'affichage graphique.

2 Objectifs finaux, Bilan des tâches et État d'avancement

Module / Responsable	Tâche (Code Rust)	Soutenance 1
Analyse Spectrale Florent	Algorithme FFT	70%
	Gestion fenêtres et magnitudes	40%
Synthèse Granulaire Oren	Archi du Buffer Circulaire	100%
	Moteur de grains	35%
Entrées/Sorties Audio Tao	Callback audio & Buffers	40%
	Archi multi-thread & Flux	50%
Système & Rendu Julien	Pipeline graphique (wgpu)	60%
	GUI finalisée & Site Web	80%

Florent

En tant que chef de groupe, mon objectif pour cette soutenance finale était de garantir la livraison d'un produit robuste et bien intégré. L'ensemble des objectifs fixés par le cahier des charges a été atteint :

- **Algorithme FFT (100 %)** : L'algorithme de Cooley-Tukey en Rust est totalement finalisé et optimisé. Il traite en place les buffers audio sans faille.
- **Gestion des fenêtres et magnitudes (100 %)** : Le calcul des magnitudes est opérationnel. L'implémentation des fonctions de fenêtrage (Hann/Hamming) a été achevée avec succès pour réduire la fuite spectrale.

Oren

Pour cette soutenance finale, l'objectif principal était d'aboutir à un moteur de synthèse granulaire totalement opérationnel, optimisé pour le temps réel et parfaitement interfacé avec la GUI et le pipeline audio. L'ensemble des objectifs fixés par le cahier des charges a été atteint à 100 % :

- **Buffer Circulaire Lock-Free (100 % - Attendu : 100 %)** : L'architecture du réservoir de son a été finalisée et éprouvée. Le tampon circulaire gère désormais la lecture et l'écriture concurrentes de manière totalement asynchrone et sécurisée via des variables atomiques, garantissant l'absence de micro-coupures sonores lors de la capture en continu.

-
- **Moteur de Synthèse Granulaire (100 % - Attendu : 100 %) :** L'algorithme de granulation (**GrainCloud**) a été considérablement enrichi. Le moteur traite désormais dynamiquement les paramètres utilisateurs : la densité d'apparition des grains, leur taille, ainsi que la transposition de hauteur. J'ai également implémenté des enveloppes de fenêtrage (lissage de l'attaque et du relâchement de chaque grain) pour supprimer définitivement les artefacts sonores ("clics") lors de la superposition.
 - **Intégration et Paramétrage (100 % - Attendu : 100 %) :** Le lien avec le module d'Entrées/Sorties de Tao et l'interface de Julien est totalement stabilisé. Les paramètres modifiés par l'utilisateur sur la GUI sont instantanément répercutés dans le moteur de synthèse sans bloquer le flux audio.

Le mois de mai a été une période charnière pour concrétiser cette intégration. Grâce à des sessions intensives avec le reste de l'équipe, nous avons pu lier le moteur granulaire aux autres modules tout en respectant notre engagement de livrer un code propre, formaté, portable (notamment sous Linux) et compilant sans aucun warning.

Tao

Au terme du projet, l'objectif principal était de disposer d'un pipeline audio totalement opérationnel, stable en temps réel et capable de relier proprement les entrées/sorties audio au moteur granulaire, à la FFT et à l'interface graphique. L'ensemble des objectifs fixés par le cahier des charges a été atteint à 100 %.

- **Callback audio & gestion des buffers (100 % - Attendu : 100 %) :** L'architecture du flux audio a été finalisée autour de **cpal**, avec une séparation claire entre callback d'entrée et callback de sortie. La capture microphone, le remplissage continu des buffers et l'alimentation du moteur audio sont désormais assurés de manière fluide, sans blocage et en respectant les contraintes du temps réel.
- **Architecture multi-thread & pilotage temps réel (100 % - Attendu : 100 %) :** La communication entre le thread principal, l'interface et le thread audio a été stabilisée grâce à l'utilisation de variables atomiques pour les paramètres temps réel (**pitch**, **density**, **grain size**, **source mode**, **effect on**, **reverse prob**) et de structures partagées ciblées pour les données plus complexes. Cette organisation permet de modifier les paramètres depuis la GUI sans bloquer le flux audio ni introduire de data races.

-
- **Gestion des sources audio et du flux applicatif (100 % - Attendu : 100 %)** : Le pipeline prend désormais en charge plusieurs modes d'utilisation : capture microphone, lecture de fichiers audio aux formats `.wav`, `.mp3` et `.flac`, enregistrement, relecture d'un buffer enregistré et export du rendu final en `.wav`. J'ai également mis en place le resampling des flux afin d'adapter proprement les différentes sources aux taux d'échantillonnage réels des périphériques audio utilisés.
 - **Intégration de la FFT dans le pipeline (100 % - Attendu : 100 %)** : Le signal de sortie audio alimente désormais un buffer d'accumulation dédié au calcul spectral. Une fois ce buffer rempli, la fenêtre de Hann, la FFT et le calcul des magnitudes en décibels sont appliqués, puis transmis à l'interface via une structure partagée, ce qui garantit un affichage spectral temps réel cohérent avec le son réellement produit.
 - **Stabilisation du comportement audio final (100 % - Attendu : 100 %)** : Les derniers ajustements ont permis de corriger le comportement du monitoring micro, d'assurer une sortie muette lorsque nécessaire et de maintenir en parallèle l'enregistrement, l'alimentation du buffer granulaire et l'analyse spectrale. Le pipeline est ainsi devenu le socle stable de l'intégration finale de l'application.

Le mois de mai a été une phase décisive pour finaliser cette architecture. Grâce au travail d'intégration mené avec le reste de l'équipe, le pipeline audio interagit désormais correctement avec le moteur granulaire, le module FFT et la GUI, tout en conservant une exécution stable, thread-safe et compatible avec les contraintes de latence du projet.

Julien

Objectifs initiaux :

- Concevoir une interface graphique native, réactive et esthétiquement soignée en Rust avec `egui/eframe`.
- Exposer tous les paramètres du moteur granulaire via des contrôles interactifs (sliders, boutons, toggles).
- Intégrer la visualisation temps réel du spectre fréquentiel produit par le module FFT.
- Développer et mettre à jour le site web de présentation du projet.
- Assurer la coordination de l'intégration finale entre les modules développés indépendamment.

Bilan :

-
- **Interface graphique (100% — Attendu : 100%)** : L'interface est entièrement fonctionnelle. Elle expose en temps réel tous les paramètres du moteur granulaire (grain, densité, pitch, source, transport, enregistrement) et intègre un analyseur de spectre FFT temps réel alimenté par le signal de sortie audio effectif. Le rendu visuel a été soigné avec un système de boutons premium, un panneau scrollable et un thème sombre cohérent.
 - **Site web (100% — Attendu : 100%)** : Le site a été conçu, développé et mis à jour de la première soutenance jusqu'à la version finale. Il présente l'architecture du projet, les défis techniques, la chronologie de réalisation, l'historique Git et les liens vers les livrables (rapport, sources, version lite), conformément aux exigences du cahier des charges.
 - **Coordination et intégration (100% — Attendu : 100%)** : Le mois de mai a été consacré à l'intégration finale entre les modules. Le pipeline FFT a été branché sur le callback audio de sortie et connecté à l'affichage graphique via un buffer partagé thread-safe (`Arc<Mutex<Option<SpectrumFrame>`). Le comportement du monitoring micro a également été corrigé pour éviter tout retour audio non désiré.

3 Difficultés & Choix technologiques et Algorithmiques

3.1 Analyse Spectrale - Florent

Le défi principal de mon module a consisté à traduire des concepts mathématiques de traitement du signal en code Rust performant, sécurisé et adapté aux très fortes contraintes de latence du domaine de l'audio. J'ai pris la décision stratégique d'implémenter le moteur d'analyse entièrement *from scratch*, sans dépendre de bibliothèques externes de calcul scientifique (comme `rustfft`).

- **Implémentation des Mathématiques Fondamentales** : Le langage Rust n'intégrant pas nativement les nombres complexes dans sa bibliothèque standard, j'ai conçu une structure `Complex` sur-mesure. L'implémentation des traits de surcharge d'opérateurs (`std::ops::Add`, `Sub`, `Mul`) m'a permis de coder les calculs d'angles polaires et de magnitudes de façon mathématiquement lisible, tout en maîtrisant parfaitement la disposition en mémoire.

-
- **Algorithme FFT En place** : L'enjeu majeur en traitement audio temps réel est d'éviter à tout prix les allocations dynamiques. J'ai donc développé un algorithme opérant directement sur les buffers via mutation (par l'intermédiaire de tranches modifiables `&mut [Complex]`). L'algorithme applique d'abord une permutation de bit, puis effectue son calcul en "papillon". Ce choix assure une complexité temporelle optimale de $\mathcal{O}(N \log N)$ et des performances extrêmes : nos tests unitaires valident l'analyse d'un buffer de 4096 échantillons en largement moins de 20 millisecondes.
 - **Pré/Post-Traitement et Fenêtrage** : Pour fournir des données directement exploitables et visuellement cohérentes pour la GUI, j'ai implémenté plusieurs étapes de lissage. Avant la FFT, une fonction de fenêtrage de Hann est appliquée sur le signal (via l'équation $\frac{1}{2}(1 - \cos(\frac{2\pi i}{N-1}))$) pour supprimer la fuite spectrale. Un mécanisme de *zero-padding* garantit que les fenêtres sont toujours des puissances de deux. Enfin, j'ai codé une conversion logarithmique des magnitudes en décibels intégrant un seuil plancher à 10^{-6} pour éviter les singularités mathématiques et refléter fidèlement la dynamique de l'oreille humaine.

3.2 Synthèse Granulaire - Oren

La difficulté principale de mon module ne résidait pas uniquement dans la logique mathématique de la granulation, mais dans la maîtrise des paradigmes stricts de concurrence du langage Rust appliqués au temps réel. Il a fallu repenser la gestion de la mémoire pour éviter toute interruption du flux audio.

- **Le défi du temps réel et l'architecture "Lock-Free"** : Pour stocker le flux continu du microphone, j'ai fait le choix de développer un *Circular Buffer* (tampon circulaire) "from scratch". La difficulté majeure était le multithreading : le thread audio (qui écrit) et le thread graphique (qui lit pour l'affichage) accèdent à la même mémoire. L'utilisation classique d'un verrou (`Mutex`) était inenvisageable : si le thread audio est bloqué en attendant que l'interface libère le verrou, il rate son échéance matérielle, provoquant des craquements sonores destructeurs. J'ai donc opté pour une architecture **Lock-Free** en utilisant des pointeurs atomiques (`Arc<AtomicUsize>`). Les indices de lecture/écriture sont mis à jour au niveau du cycle processeur, garantissant une synchronisation asynchrone parfaite et une latence minimale.
- **Architecture de l'Overlap-Add et lissage granulaire** : La conception du système `GrainCloud` a nécessité une gestion précise du cycle

de vie des grains. Le moteur effectue la sommation des échantillons de plusieurs grains simultanés pour générer la sortie audio en temps réel. Une des difficultés rencontrées lors des premiers tests était l'apparition de "clics" (artefacts audio) dus aux coupures abruptes du signal lors du prélèvement des grains. Pour pallier ce problème, j'ai implémenté l'application d'une courbe de Hanning (fenêtrage) sur chaque grain, lissant l'attaque et le relâchement, ce qui rend la texture sonore finale parfaitement fluide, même lors de transpositions extrêmes de hauteur ou de densité.

- **Rigueur de compilation et interopérabilité Linux :** Un défi inattendu s'est présenté lors de la phase de déploiement final. Notre cahier des charges exigeait une compatibilité stricte sous Linux, ainsi qu'une compilation sans aucun avertissement (*warning*). Bien que le code compilait parfaitement sur macOS et Windows, les tests sous Debian ont échoué en raison de la sensibilité à la casse du système de fichiers de Linux (fichiers originaux nommés `FFT.rs`). J'ai dû intervenir sur l'index de notre gestionnaire de version en forçant le renommage des fichiers via le terminal (`git mv`) pour respecter scrupuleusement la norme *snake_case* (`fft.rs`) imposée par Rust. Cette rigueur nous a permis de valider définitivement le critère du "zéro warning"

3.3 Entrées/Sorties Audio & Pipeline - Tao

La difficulté principale de mon module ne résidait pas dans la correction de bugs bloquants, mais dans la structuration d'un pipeline audio temps réel cohérent avec les contraintes du callback imposé par la bibliothèque audio. Il a fallu organiser proprement la circulation des données entre les entrées, les traitements et la sortie, tout en garantissant une exécution stable, légère et sans interruption sonore.

- **Comprendre le modèle callback et respecter les contraintes du temps réel :** Le premier enjeu a été de concevoir le code autour du fonctionnement imposé par `cpal`, où le système appelle régulièrement une fonction de rappel pour demander de nouveaux échantillons audio. Dans ce modèle, le callback doit rester extrêmement léger, déterministe et exempt d'opérations coûteuses ou bloquantes, sous peine de provoquer des coupures sonores. J'ai donc structuré le pipeline de manière à séparer clairement la capture, la lecture des buffers, l'application des traitements et l'écriture vers la sortie audio, afin de garantir un fonctionnement fluide et compatible avec les contraintes du temps réel.

-
- **Pilotage des paramètres et synchronisation inter-thread :** Une difficulté importante concernait la communication entre le thread principal, qui pilote les paramètres depuis l'interface, et le thread audio, qui doit rester prioritaire et ne jamais être bloqué. L'utilisation de verrous classiques dans le callback était à éviter, car elle aurait pu perturber la régularité du flux audio. J'ai donc retenu une architecture reposant principalement sur des variables atomiques partagées (`Arc<AtomicU32>`, `Arc<AtomicBool>`, `Arc<AtomicUsize>`) pour transmettre les paramètres et les états de contrôle en toute sécurité. Ce choix permet de modifier les réglages en temps réel sans introduire de data races ni compromettre la stabilité sonore.
 - **Gestion du flux audio et intégration des différentes sources :** Le pipeline devait prendre en charge plusieurs cas d'usage : microphone, lecture de fichiers audio, enregistrement, relecture et export final. J'ai donc mis en place une architecture capable de charger des fichiers `.wav`, `.mp3` et `.flac`, de convertir les données en mono si nécessaire, puis de les adapter au taux d'échantillonnage réel de sortie grâce à un resampling linéaire. Cette étape était essentielle pour garantir une lecture correcte des différentes sources au sein d'un même moteur audio, tout en conservant une chaîne cohérente entre entrée, traitement et sortie.
 - **Intégration du buffer de traitement dans le pipeline :** Même si le cœur du traitement granulaire est développé dans un autre module, il a fallu intégrer dès le pipeline les structures nécessaires pour alimenter correctement ce moteur. L'ajout du buffer circulaire dans la chaîne audio permet de conserver un historique des échantillons et de transmettre en continu les données utiles aux traitements basés sur le signal passé. Ce choix a permis de valider très tôt le bon fonctionnement de la chaîne complète Entrée → Traitement → Sortie, tout en préparant l'intégration du moteur granulaire dans des conditions proches de l'exécution finale.
 - **Branchement de la FFT et stabilisation du comportement global :** Une autre étape importante a consisté à intégrer l'analyse spectrale directement dans le pipeline de sortie, afin que la FFT reflète bien le signal réellement produit par l'application. J'ai mis en place un buffer d'accumulation local alimenté dans le callback audio, utilisé ensuite pour calculer la FFT et transmettre les résultats à l'interface graphique. Enfin, certains comportements inattendus, comme le retour audio non désiré du microphone, ont nécessité des ajustements pour conserver un système propre, prévisible et stable. Ces choix ont permis d'aboutir à un pipeline fiable, capable de servir de socle à l'intégration

finale de l'ensemble du projet.

3.4 Système & Rendu - Julien

Synchronisation entre le thread audio et le thread graphique Le principal défi technique de ma partie a été la communication entre le thread audio temps réel — géré par `cpal` dans un callback non bloquant — et le thread graphique d'egui, qui tourne dans la boucle principale de l'application.

Problème : Un accès direct à des structures partagées entre les deux threads aurait provoqué des data races, incompatibles avec les garanties de sécurité de Rust et potentiellement sources de crackling audio ou de plantages.

Solution : Utilisation systématique de types atomiques (`Arc<AtomicU32>`, `Arc<AtomicBool>`, `Arc<AtomicUsize>`) pour tous les paramètres simples (pitch, densité, taille de grain, flags d'état), et de `Arc<Mutex<T>` uniquement pour les structures plus complexes nécessitant un accès exclusif (buffer d'enregistrement, frame FFT partagée). Cette architecture garantit une communication thread-safe sans verrou bloquant dans le callback audio.

Intégration du pipeline FFT et affichage temps réel **Problème :** Le module FFT, développé indépendamment, devait être branché sur le signal de sortie audio réel, ses résultats transmis au GUI sans introduire de latence ni de blocage dans le callback audio.

Solution : Ajout d'un buffer d'accumulation local dans le callback audio (`Vec<f32>` de 1024 samples). Lorsque ce buffer est plein, la FFT est calculée (avec fenêtre de Hann), le résultat est écrit dans une structure partagée de type `Arc<Mutex<Option<SpectrumFrame>>`, puis le GUI lit ce buffer à chaque frame de rendu via la commande `self.shared_spectrum.lock()`. Cette approche découple proprement le rythme de calcul FFT du rythme de rendu graphique.

Comportement inattendu du monitoring micro **Problème :** En mode source micro, le signal capturé était directement renvoyé en sortie audio, provoquant un retour casque non désiré même sans action de l'utilisateur.

Solution : Modification de la branche micro du callback audio pour que la sortie soit systématiquement muette (`out = 0.0`), tout en conservant l'alimentation du buffer granulaire et l'enregistrement conditionnel via `recording_flag`. La FFT est calculée sur `live_sample` dans cette branche pour maintenir un spectre réactif même sans monitoring actif.

4 Livrables Finaux et Historique de Développement

Conformément au cahier des charges de la soutenance finale, notre projet s'accompagne des éléments de livraison suivants :

- **Aide à la compilation et exécution** : Un fichier `README.md` exhaustif est inclus à la racine de notre dépôt. Il détaille les prérequis systèmes sous Linux, les commandes Cargo nécessaires à la compilation de notre package unique, ainsi que les instructions d'installation des dépendances audio système.
- **Visualisation de l'historique (Git)** : Tout au long des cinq mois de développement, nous avons maintenu une utilisation rigoureuse de notre gestionnaire de version. Une visualisation de l'ensemble de nos *commits* a été générée via l'outil Gource (fournie sous forme de vidéo dans les livrables du site web), permettant au jury d'observer de manière transparente la chronologie de réalisation et la répartition de la charge de travail entre les quatre membres.
- **Site Web et Binaire** : Le binaire exécutable, ses dépendances, ainsi qu'une version "lite" allégée, sont directement téléchargeables depuis notre plateforme web dédiée.

5 Bibliographie Détaillée du Projet

La réalisation de ce projet s'appuie sur un corpus documentaire structuré, allant des mathématiques de l'analyse spectrale à la programmation système en Rust.

1. Théorie du Traitement du Signal

- KUNT, Murat. *Traitement numérique des signaux*. Presses polytechniques et universitaires romandes. (Fondations sur l'échantillonnage et la décomposition de Fourier).
- COTTET, Francis. *Traitement des signaux et acquisition de données*. Dunod. (Guide sur l'implémentation des filtres et de la FFT).
- OKSMAN, Jacques. *Théorie et traitement du signal*. Dunod. (Étude des fonctions de fenêtrage).

2. Informatique Musicale et Synthèse Sonore

- ROADS, Curtis. *L'audio numérique*. Dunod. (Ouvrage de référence sur la synthèse granulaire et la technique de l'Overlap-Add).

-
- VINET, Hugues. *Le traitement du signal sonore*. Hermès Science. (Transformation spectrale appliquée à la création musicale).

3. Programmation Système et Algorithmique

- BLANDY, Jim et ORENDORFF, Jason. *Programmation en Rust*. O'Reilly. (Maîtrise de l'Ownership et du multithreading lock-free).
- SEDGEWICK, Robert. *Algorithmes en C / C++ / Rust*. Pearson. (Optimisation des structures de données dynamiques).

4. Bibliothèques et Standards Techniques

- DOCUMENTATION OFFICIELLE RUST. <https://doc.rust-lang.org/book/>. (Gestion des cycles de vie pour les tampons audio partagés).
- CPAL CORE TEAM. *CPAL - Technical Documentation*. <https://docs.rs/cpal/>. (Configuration bas niveau d'ALSA et callback temps réel).
- EGUI CORE TEAM. *Egui - API Reference*. <https://docs.rs/egui/>. (Conception de l'interface en Immediate Mode et rendu à 60 FPS).

Cette bibliographie a garanti que le moteur **SPECTRALIS** repose sur des bases mathématiques solides tout en exploitant les outils de l'écosystème Rust.